

Enterprise MLOps End-to-End on GCP

UI-Only Lab Workshop — Consolidated Learner Guide

Contents

How to Use This Guide	2
Before You Start — Prerequisites Checklist	2
SECTION 1 — GCP Environment Setup and ML Deployment	3
Step 1.1 — Create the Project	3
Step 1.2 — Link Billing and Set a Budget	3
Step 1.3 — Enable Required APIs	4
Step 1.4 — Set Default Region	5
Step 1.5 — Create the Cloud Storage Bucket and Folders	5
Step 1.6 — Prepare the Dataset and Train the Model Locally	6
Step 1.7 — Upload Data and Model to Cloud Storage (via Console)	8
Step 1.8 — Write the FastAPI App, Dockerfile, and Requirements	8
Step 1.9 — Push Code to GitHub (via GitHub’s Web UI)	9
Step 1.10 — Deploy to Cloud Run (Connected to GitHub, via Console)	9
Step 1.11 — Test the Deployed Service	10
Step 1.12 — Generate Production Predictions (Batch)	11
SECTION 2 — Production Predictions, Monitoring, and Experiment Tracking	13
Step 2.1 — Explore Cloud Logging and Monitoring	13
Step 2.2 — Generate a Drifted Dataset for the Monitoring Demo	14
Step 2.3 — Run Drift Detection Locally and Upload the Report	14
Step 2.4 — Experiment Tracking with MLflow (Local UI)	15
SECTION 3 — Retraining, Model Lifecycle, and Governance	17
Step 3.1 — Retrain and Compare (Champion vs. Challenger)	17
Step 3.2 — Upload the New Model and Deploy v2 (via GitHub + Cloud Run)	18
Step 3.3 — Review Revisions and Roll Back (via Console)	18
Step 3.4 — Governance and IAM Review (via Console)	19
SECTION 4 — Cost, ROI, Automation, and Enterprise Architecture	20
Step 4.1 — Review Costs in Cloud Billing (Console)	20
Step 4.2 — Cost Estimate and ROI Calculation	20
Step 4.3 — Automate Scheduled Predictions (Console)	21
Step 4.4 — Present the Full Architecture	22
SECTION 5 — Conclusion, Cleanup, and Q&A	24
Step 5.1 — Recap Checklist	24
Step 5.2 — Mandatory Cleanup (via Console — Do Not Skip)	24
Step 5.3 — Open Q&A	24

How to Use This Guide

This is the **consolidated learner guide** for the 3-hour Enterprise MLOps workshop on Google Cloud Platform. It merges two sources into one document:

1. **The step-by-step lab instructions** — every command, script, and click needed to complete the workshop, using only the Google Cloud Console and GitHub’s web UI (no `gcloud` / CLI required).
2. **The visual walkthrough** — a schematic wireframe of the relevant Console screen, dropped in immediately after the step it illustrates, under a ► **Where to perform this** heading.

A note on the diagrams: these are wireframes drawn to match the layout and wording of the Google Cloud Console (top bar, left navigation, main panel) — not literal screenshots. Google updates its Console UI regularly, so exact colors, spacing, and button placement may differ slightly on the day you run the lab. Numbered red circles correspond to the numbered instructions beside each image. **Always follow the on-screen labels in your real Console as the source of truth.**

Business problem used throughout this lab: Customer Churn Prediction — predicting which telecom customers are likely to cancel their subscription.

Constraints this lab respects:

- Region: asia-south1 (Mumbai) | Python: 3.11 | CPU-only
- Model: Logistic Regression or Decision Tree only, `.joblib`, ≤ 5 MB
- Historical dataset ≤ 25 MB, ≤ 5,000 rows, ≤ 15 features
- Prediction dataset: 1,000 rows, ≤ 5 MB, same schema minus target
- Cloud Run: 1 vCPU, 2 GB RAM

How code gets deployed without the CLI: we use Cloud Run’s built-in “**Continuously deploy from a repository**” option, which connects to a GitHub repo, auto-detects your Dockerfile, and builds + deploys entirely through the Console UI (Cloud Build runs behind the scenes automatically — no manual `docker build` / `gcloud` needed).

Replace placeholders like `YOUR_PROJECT_ID`, `your-bucket-name`, and `your-github-username` with your own values throughout.

Before You Start — Prerequisites Checklist

- ☐ Google account with billing enabled
- ☐ A free GitHub account (`github.com`) — used to host your code so Cloud Run can deploy from it
- ☐ Python 3.11 installed locally, to write and test scripts before uploading them (or use **Cloud Shell Editor**, a browser-based code editor inside the GCP Console — click the **Open Editor** icon in the Cloud Shell terminal bar; this is still “UI,” you never type `gcloud` commands)

SECTION 1 — GCP Environment Setup and ML Deployment

Step 1.1 — Create the Project

1. Go to **console.cloud.google.com**
2. Click the project dropdown (top left, next to “Google Cloud”) → **New Project**
3. Name it MLOps Churn Lab, note the generated Project ID (or set a custom one)
4. Click **Create**, then select the new project from the dropdown once it’s ready

► **Where to perform this — Select a project → New Project**

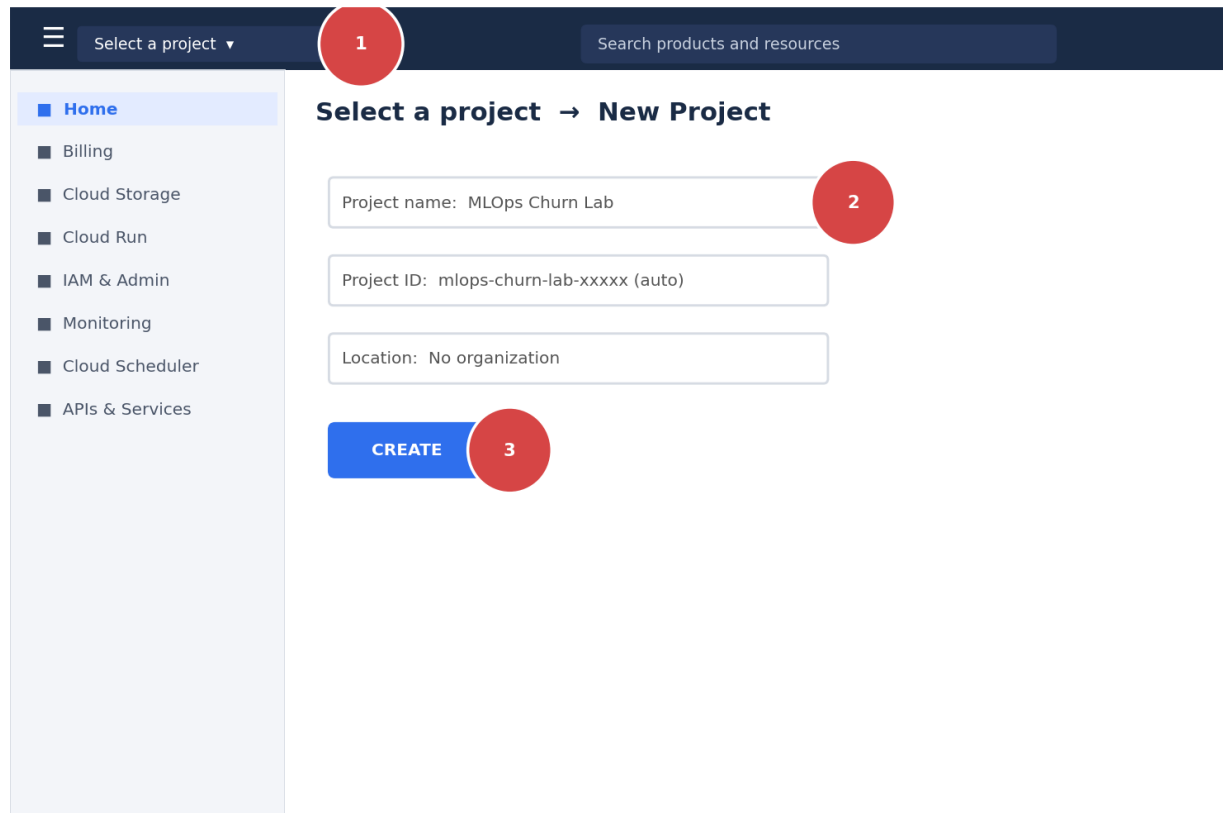


Figure 1: Create Project wireframe

Step 1.2 — Link Billing and Set a Budget

1. Left sidebar ([menu icon]) → **Billing**
2. If prompted, **Link a billing account** to this project
3. Go to **Billing → Budgets & alerts → Create Budget**
4. Name it Churn Lab Budget, set scope to this project only
5. Set an amount (e.g., ₹500 / \$10)
6. Set alert thresholds at 50%, 90%, 100% of budget → **Finish**

► **Where to perform this — Billing → Budgets & alerts → Create Budget**

The wireframe shows the 'Create Budget' page in the Google Cloud console. The left sidebar contains a menu with 'Billing' highlighted. The main content area has a breadcrumb trail: 'Billing → Budgets & alerts → Create Budget'. The form consists of five numbered steps: 1. 'Name: Churn Lab Budget', 2. 'Scope: This project only', 3. 'Target amount: ₹500 / \$10', 4. 'Alert thresholds' with radio buttons for '50%', '90%', and '100% of budget', and 5. A blue 'FINISH' button.

Figure 2: Budget and Alerts wireframe

Step 1.3 — Enable Required APIs

1. Left sidebar → **APIs & Services** → **Library**
2. Search for and click **Enable** on each of the following, one at a time:
 - Cloud Run Admin API
 - Artifact Registry API
 - Cloud Storage API
 - Cloud Logging API
 - Cloud Monitoring API
 - Cloud Scheduler API
 - Identity and Access Management (IAM) API
 - Cloud Build API

► **Where to perform this — APIs & Services → Library**

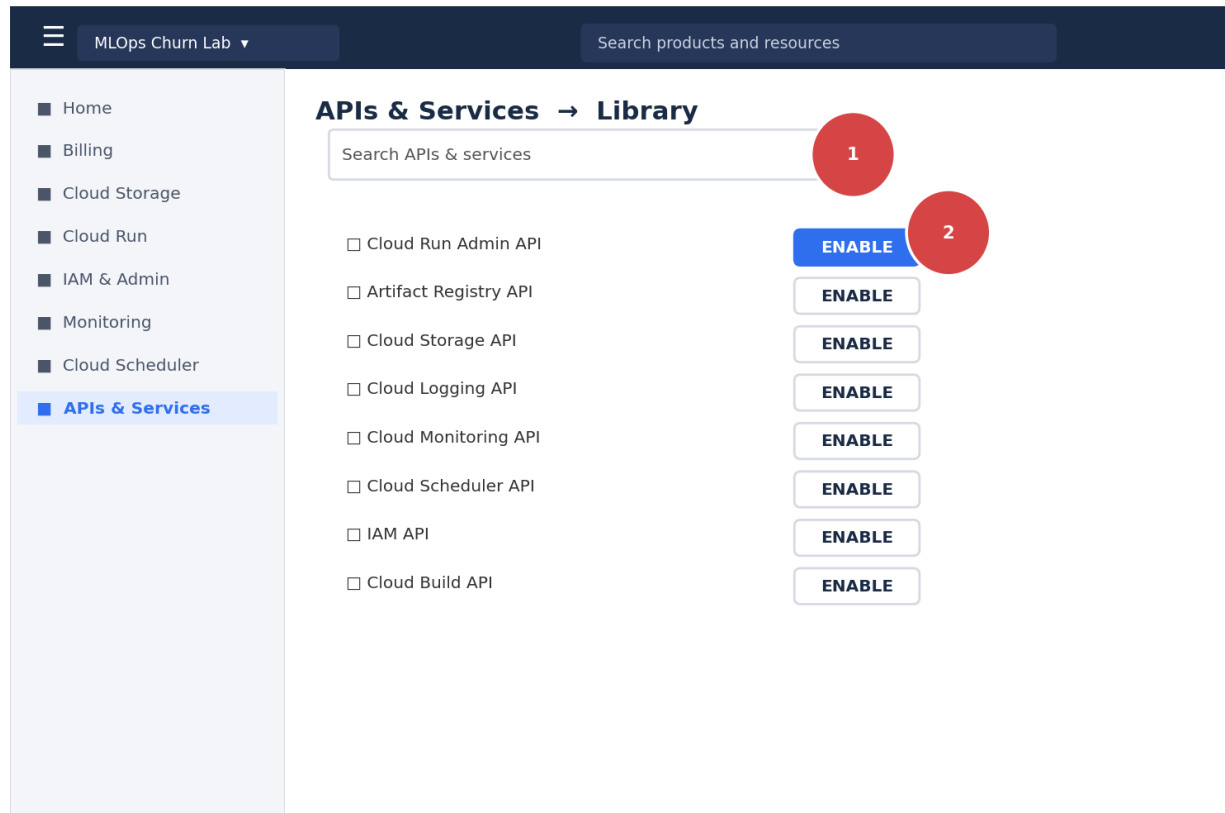


Figure 3: Enable APIs wireframe

Step 1.4 — Set Default Region

1. Left sidebar → **Cloud Run** → click **Settings** (or the region selector at top of the Cloud Run page)
2. Set default region to **asia-south1 (Mumbai)** — you'll select this again per-service, but set it as default to save clicks

Step 1.5 — Create the Cloud Storage Bucket and Folders

1. Left sidebar → **Cloud Storage** → **Buckets** → **Create**
2. Name: your-bucket-name (must be globally unique)
3. Location type: **Region** → select **asia-south1 (Mumbai)**
4. Storage class: **Standard**
5. Access control: **Uniform**
6. Click **Create**
7. Inside the bucket, click **Create folder** and add each of these, one at a time:
 - training-data
 - prediction-data
 - models
 - predictions
 - logs
 - artifacts

► **Where to perform this — Cloud Storage → Buckets → Create**

Figure 4: Cloud Storage Bucket wireframe

Step 1.6 — Prepare the Dataset and Train the Model Locally

Write these files locally (in any code editor) or in **Cloud Shell Editor**:

generate_data.py:

```
import pandas as pd
import numpy as np

np.random.seed(42)
n = 4000

df = pd.DataFrame({
    'tenure_months': np.random.randint(1, 72, n),
    'monthly_charges': np.round(np.random.uniform(20, 120, n), 2),
    'total_charges': np.round(np.random.uniform(20, 8000, n), 2),
    'contract_type': np.random.choice([0, 1, 2], n),
    'internet_service': np.random.choice([0, 1, 2], n),
    'tech_support': np.random.choice([0, 1], n),
    'online_security': np.random.choice([0, 1], n),
    'paperless_billing': np.random.choice([0, 1], n),
    'payment_method': np.random.choice([0, 1, 2, 3], n),
    'num_support_calls': np.random.poisson(1.5, n),
    'senior_citizen': np.random.choice([0, 1], n),
    'partner': np.random.choice([0, 1], n),
})
```

```
        'dependents':          np.random.choice([0, 1], n),
    })

    churn_score = (
        -0.03 * df['tenure_months']
        + 0.02 * df['monthly_charges']
        - 0.8 * df['contract_type']
        + 0.5 * df['num_support_calls']
        - 0.6 * df['tech_support']
        - 0.4 * df['online_security']
        + np.random.normal(0, 1, n)
    )
    df['churn'] = (churn_score > np.median(churn_score)).astype(int)

    train_df = df.iloc[:3000].copy()
    train_df.to_csv('historical_data.csv', index=False)

    pred_df = df.iloc[3000:].drop(columns=['churn']).reset_index(drop=True)
    pred_df.to_csv('prediction_data.csv', index=False)

    print('historical_data.csv:', train_df.shape)
    print('prediction_data.csv:', pred_df.shape)
```

Run this locally: `python generate_data.py` (this one step needs a plain Python run — it's just generating a CSV, not a cloud action).

train_model.py:

```
import pandas as pd
import joblib
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

df = pd.read_csv('historical_data.csv')

EXPECTED_COLS =
    ↪ ['tenure_months', 'monthly_charges', 'total_charges', 'contract_type',
        'internet_service', 'tech_support', 'online_security',
        ↪ 'paperless_billing',
        'payment_method', 'num_support_calls', 'senior_citizen', 'partner',
        'dependents', 'churn']
assert list(df.columns) == EXPECTED_COLS, "Schema mismatch!"
assert df.isnull().sum().sum() == 0, "Missing values found!"
print("Schema validation passed.")

FEATURES = [c for c in EXPECTED_COLS if c != 'churn']
X, y = df[FEATURES], df['churn']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42, stratify=y)

model = LogisticRegression(max_iter=1000, class_weight='balanced')
model.fit(X_train, y_train)
```

```
preds = model.predict(X_val)
print(f"Accuracy: {accuracy_score(y_val, preds):.3f}")
print(f"F1 Score: {f1_score(y_val, preds):.3f}")

joblib.dump({'model': model, 'features': FEATURES, 'version': 'v1'},
            ↪ 'churn_model.joblib')
print("Model saved as churn_model.joblib")
```

Run locally: `python train_model.py`. Confirm `churn_model.joblib` is under 5 MB by checking its file size in your file explorer.

Step 1.7 — Upload Data and Model to Cloud Storage (via Console)

1. Go to **Cloud Storage** → **Buckets** → **your-bucket-name**
2. Open the training-data folder → **Upload files** → select `historical_data.csv`
3. Go back, open prediction-data folder → **Upload files** → select `prediction_data.csv`
4. Go back, open models folder → **Upload files** → select `churn_model.joblib` (rename it to `churn_model_v1.joblib` after upload using the **Rename** option in the file's [...] menu, or rename it locally before uploading)

(This reuses the same bucket UI shown in Step 1.5 above — just switch folders.)

Step 1.8 — Write the FastAPI App, Dockerfile, and Requirements

`main.py`:

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import pandas as pd
from datetime import datetime

app = FastAPI(title="Churn Prediction API")

MODEL_PATH = "churn_model.joblib"
bundle = joblib.load(MODEL_PATH)
model, FEATURES, VERSION = bundle['model'], bundle['features'], bundle['version']

class ChurnRequest(BaseModel):
    tenure_months: int
    monthly_charges: float
    total_charges: float
    contract_type: int
    internet_service: int
    tech_support: int
    online_security: int
    paperless_billing: int
    payment_method: int
    num_support_calls: int
    senior_citizen: int
    partner: int
    dependents: int

@app.get("/health")
def health():
```



```
    return {"status": "ok", "timestamp": datetime.utcnow().isoformat()}

@app.get("/metadata")
def metadata():
    return {"model_version": VERSION, "features": FEATURES}

@app.post("/predict")
def predict(req: ChurnRequest):
    row = pd.DataFrame([req.dict()])[FEATURES]
    pred = int(model.predict(row)[0])
    prob = float(model.predict_proba(row)[0][1])
    return {
        "prediction": pred,
        "churn_probability": round(prob, 4),
        "model_version": VERSION,
        "timestamp": datetime.utcnow().isoformat()
    }
```

requirements.txt:

```
fastapi==0.111.0
uvicorn[standard]==0.30.1
scikit-learn==1.5.0
pandas==2.2.2
joblib==1.4.2
```

Dockerfile:

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY main.py churn_model.joblib ./
```

```
EXPOSE 8080
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

(Optional but recommended: test the API locally by running `uvicorn main:app --reload --port 8080` and hitting `http://localhost:8080/docs` in your browser — FastAPI's built-in interactive UI, so you can test /predict by filling in a form, no curl needed.)

Step 1.9 — Push Code to GitHub (via GitHub's Web UI)

1. Go to **github.com** → **New repository** → name it `churn-mlops-lab` → **Create repository**
2. Click **uploading an existing file** on the empty repo page
3. Drag and drop `main.py`, `requirements.txt`, `Dockerfile`, and `churn_model.joblib` into the upload box
4. Scroll down, click **Commit changes**

Step 1.10 — Deploy to Cloud Run (Connected to GitHub, via Console)

1. Go to **Cloud Run** → **Create Service**
2. Select **Continuously deploy new revisions from a source repository**
3. Click **Set up with Cloud Build**

4. Choose **GitHub** as the source, click **Authenticate** and authorize GCP to access your GitHub account
5. Select the repository `churn-mlops-lab`
6. Branch: `main`, Build type: **Dockerfile** (auto-detected), Dockerfile location: `/Dockerfile`
7. Click **Save**
8. Back on the Create Service screen, set:
 - Service name: `churn-api`
 - Region: **asia-south1 (Mumbai)**
 - Authentication: **Allow unauthenticated invocations**
 - CPU: **1**, Memory: **2 GiB** (under **Container, Networking, Security** → **Container** → **Resources**)
9. Click **Create**

Cloud Build will build the image and deploy it automatically. Wait for the green checkmark, then copy the **Service URL** shown at the top of the service page (e.g., `https://churn-api-xxxxx-el.a.run.app`).

► **Where to perform this — Cloud Run → Create Service**

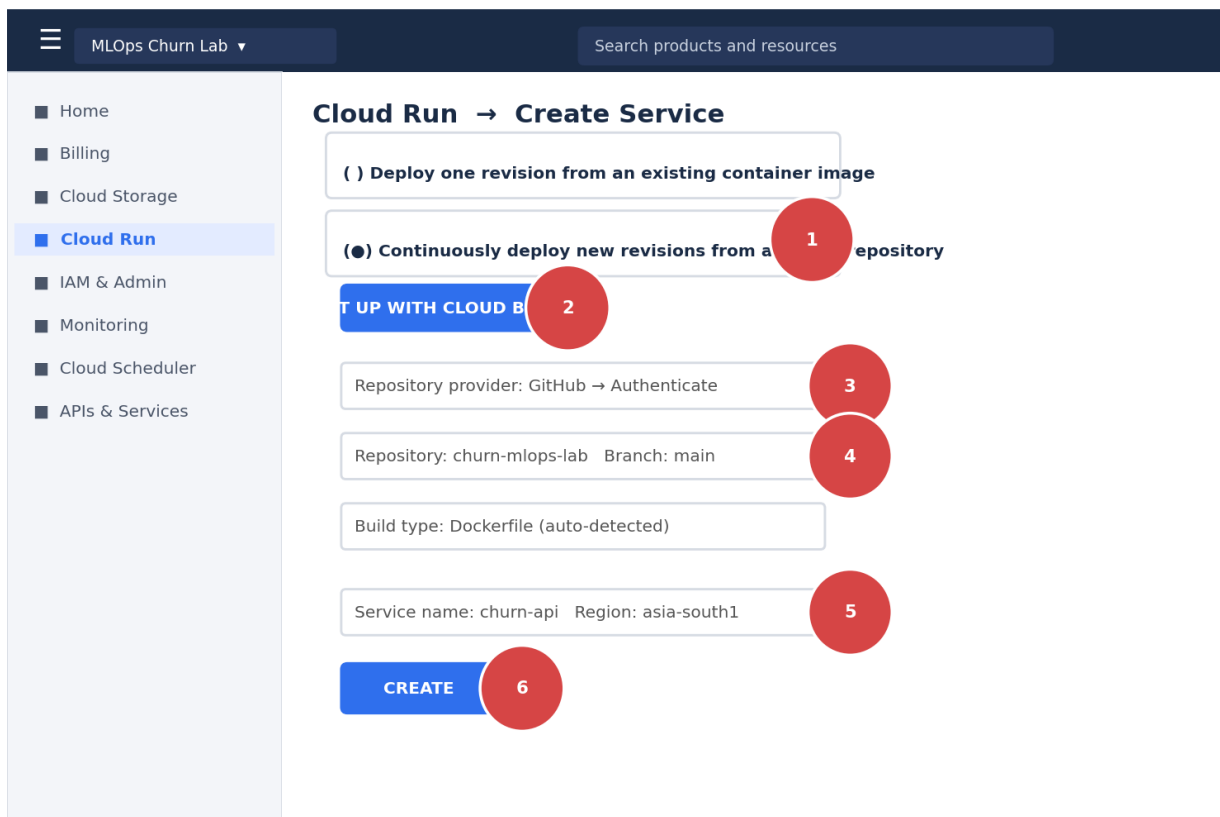


Figure 5: Cloud Run deploy from GitHub wireframe

This is the step that replaces every `gcloud` / `docker` CLI command with a Console wizard.

Step 1.11 — Test the Deployed Service

1. Open the Service URL + `/docs` in a browser (e.g., `https://churn-api-xxxxx-el.a.run.app/docs`) — this loads FastAPI's interactive Swagger UI

2. Click **GET /health** → **Try it out** → **Execute** — confirm you get {"status": "ok", ...}
3. Click **GET /metadata** → **Try it out** → **Execute** — confirm model_version: v1
4. Click **POST /predict** → **Try it out**, fill in sample values in the form, click **Execute** — confirm you get a prediction back

Step 1.12 — Generate Production Predictions (Batch)

Add a batch endpoint so the whole flow (read from GCS → predict → write to GCS) runs on Cloud Run itself, no local script needed.

Add to main.py (append this, then re-upload to GitHub — Cloud Run redeploys automatically):

```
from google.cloud import storage
import io

@app.post("/run-batch-predict")
def run_batch_predict():
    client = storage.Client()
    bucket = client.bucket("your-bucket-name")

    blob_in = bucket.blob("prediction-data/prediction_data.csv")
    data = blob_in.download_as_text()
    df = pd.read_csv(io.StringIO(data))

    results = []
    for _, row in df.iterrows():
        row_df = pd.DataFrame([row.to_dict()])[FEATURES]
        pred = int(model.predict(row_df)[0])
        prob = float(model.predict_proba(row_df)[0][1])
        results.append(**row.to_dict(), "prediction": pred, "churn_probability":
        ↪ prob,
                                "model_version": VERSION, "timestamp":
                                ↪ datetime.utcnow().isoformat())

    out_df = pd.DataFrame(results)
    filename = f"predictions_{datetime.utcnow().strftime('%Y%m%d_%H%M%S')}.csv"
    bucket.blob(f"predictions/{filename}").upload_from_string(out_df.to_csv(index
    ↪ =False),
    ↪ content_type="text/csv")
    return {"status": "done", "file": filename, "rows": len(out_df)}
```

Add google-cloud-storage==2.16.0 to requirements.txt.

1. Go to your GitHub repo → open main.py → click the pencil icon to edit directly in the browser
2. Paste in the updated code → **Commit changes** (Cloud Run will auto-rebuild and redeploy)
3. Also edit requirements.txt the same way to add the new dependency → **Commit changes**
4. Wait for Cloud Run's **Revisions** tab to show a new green revision
5. Go back to /docs on your service URL → **POST /run-batch-predict** → **Try it out** → **Execute**
6. Go to **Cloud Storage** → **your-bucket-name** → **predictions/** — confirm a new CSV appeared

Note on permissions: Cloud Run's default service account usually has Storage

access within the same project already. If the batch call fails with a permissions error, go to **IAM & Admin** → **IAM**, find the Cloud Run service's identity (usually PROJECT_NUMBER-compute@developer.gserviceaccount.com), click the pencil icon, and add the role **Storage Object Admin**.

[DONE] Checkpoint — Section 1 complete when: the service is live on Cloud Run, /health and /metadata work in the Swagger UI, and a predictions CSV (with timestamp + model version) is sitting in predictions/ in your bucket.

SECTION 2 — Production Predictions, Monitoring, and Experiment Tracking

Step 2.1 — Explore Cloud Logging and Monitoring

1. Go to **Cloud Run** → **churn-api** → **Logs tab** — view live request logs
2. Go to **Cloud Run** → **churn-api** → **Metrics tab** — view request count, latency, container CPU/memory utilization, and error rate charts, already built in
3. Go to **Monitoring** → **Dashboards** → click **Create Dashboard**, name it Churn API Ops
4. Add a chart: select resource type **Cloud Run Revision**, metric **Request Count**; add a second chart for **Request Latency**
5. Go to **Monitoring** → **Alerting** → **Create Policy**
 - Metric: Cloud Run → **Request Count**, filter by response code class 5xx
 - Condition: threshold above 5 in 5 minutes
 - Notification channel: add your email
 - Name the policy High Error Rate Alert → **Create**

► **Where to perform this — Monitoring → Dashboards / Alerting**

Figure 6: Monitoring Dashboard and Alert wireframe

Key distinction: infrastructure monitoring (this step) vs. ML monitoring (next step) — a service can be perfectly healthy on CPU/latency while its predictions are quietly drifting.

Step 2.2 — Generate a Drifted Dataset for the Monitoring Demo

`generate_drift_data.py` (run locally):

```
import pandas as pd
import numpy as np

np.random.seed(7)
n = 500

drifted = pd.DataFrame({
    'tenure_months': np.random.randint(1, 72, n),
    'monthly_charges': np.round(np.random.uniform(60, 180, n), 2),
    'total_charges': np.round(np.random.uniform(20, 8000, n), 2),
    'contract_type': np.random.choice([0, 1, 2], n, p=[0.7, 0.2, 0.1]),
    'internet_service': np.random.choice([0, 1, 2], n),
    'tech_support': np.random.choice([0, 1], n),
    'online_security': np.random.choice([0, 1], n),
    'paperless_billing': np.random.choice([0, 1], n),
    'payment_method': np.random.choice([0, 1, 2, 3], n),
    'num_support_calls': np.random.poisson(4.0, n),
    'senior_citizen': np.random.choice([0, 1], n),
    'partner': np.random.choice([0, 1], n),
    'dependents': np.random.choice([0, 1], n),
})
drifted.to_csv('drifted_production_data.csv', index=False)
print("drifted_production_data.csv created:", drifted.shape)
```

Upload `drifted_production_data.csv` to **Cloud Storage** → **your-bucket-name** → **prediction-data/** using the Console's **Upload files** button (keep the original `prediction_data.csv` there too — you'll compare both).

(Reuses the same bucket UI shown in Step 1.5 — just navigate to `prediction-data/` instead of `training-data/`.)

Step 2.3 — Run Drift Detection Locally and Upload the Report

`drift_check.py` (run locally — this analysis step doesn't need to run on Cloud Run):

```
import pandas as pd
from evidently.legacy.report import Report
from evidently.legacy.metric_preset import DataDriftPreset

reference = pd.read_csv('historical_data.csv').drop(columns=['churn'])
healthy = pd.read_csv('prediction_data.csv')
drifted = pd.read_csv('drifted_production_data.csv')

report_healthy = Report(metrics=[DataDriftPreset()])
report_healthy.run(reference_data=reference, current_data=healthy)
report_healthy.save_html('drift_report_healthy.html')
r1 = report_healthy.as_dict()['metrics'][0]['result']
print(f"[Healthy batch] Drift detected: {r1['dataset_drift']} | "
      f"Drifted features: "
      f"↪ {r1['number_of_drifted_columns']}/{r1['number_of_columns']}")

report_drifted = Report(metrics=[DataDriftPreset()])
```

```
report_drifted.run(reference_data=reference, current_data=drifted)
report_drifted.save_html('drift_report_drifted.html')
r2 = report_drifted.as_dict()['metrics'][0]['result']
print(f"[Drifted batch] Drift detected: {r2['dataset_drift']} | "
      f"Drifted features:
      ↪ {r2['number_of_drifted_columns']}/{r2['number_of_columns']}")
```

Run: `python drift_check.py`. This produces two HTML files.

1. Go to **Cloud Storage** → **your-bucket-name** → **logs/** → **Upload files** → select both `drift_report_healthy.html` and `drift_report_drifted.html`
2. Click each uploaded file in the Console → **Download** (or open the public URL if the bucket allows it) to view the report in your browser

Discuss with learners: compare the two reports side by side. The healthy batch should show 0 or minimal drift; the drifted batch should flag `monthly_charges`, `contract_type`, and `num_support_calls`.

Mentioned conceptually only: **Vertex AI Model Monitoring** (found under **Vertex AI** → **Model Monitoring** in the Console) could automate this comparison continuously in production — show learners where it lives in the menu, no need to configure it.

Step 2.4 — Experiment Tracking with MLflow (Local UI)

`track_experiment.py` (run locally):

```
import mlflow
import mlflow.sklearn
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("Churn_Model_Tracking")

df = pd.read_csv('historical_data.csv')
FEATURES = [c for c in df.columns if c != 'churn']
X, y = df[FEATURES], df['churn']
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42, stratify=y)

candidates = {
    'LogisticRegression': LogisticRegression(max_iter=1000,
    ↪ class_weight='balanced'),
    'DecisionTree': DecisionTreeClassifier(max_depth=5, class_weight='balanced',
    ↪ random_state=42),
}

for name, model in candidates.items():
    with mlflow.start_run(run_name=name):
        model.fit(X_train, y_train)
        preds = model.predict(X_val)
        acc, f1 = accuracy_score(y_val, preds), f1_score(y_val, preds)
```

```
mlflow.log_param('model_type', name)
mlflow.log_metric('accuracy', round(acc, 4))
mlflow.log_metric('f1_score', round(f1, 4))
mlflow.sklearn.log_model(model, 'model')
print(f"{name}: accuracy={acc:.3f}, f1={f1:.3f}")
```

Run: `python track_experiment.py`, then run `mlflow ui --port 5000` and open **`http://localhost:5000`** in your browser — this is MLflow's own web UI. Click into each run to show learners the logged params, metrics, and model artifact, entirely by clicking through the interface.

[DONE] Checkpoint — Section 2 complete when: the Cloud Monitoring dashboard and alert exist, two drift HTML reports (healthy + drifted) are uploaded to logs/, and MLflow's web UI shows two tracked runs.

SECTION 3 — Retraining, Model Lifecycle, and Governance

Step 3.1 — Retrain and Compare (Champion vs. Challenger)

`retrain_model.py` (run locally):

```
import pandas as pd
import joblib
import mlflow
import mlflow.sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

historical = pd.read_csv('historical_data.csv')
new_data = pd.read_csv('drifted_production_data.csv').copy()
new_data['churn'] = (new_data['num_support_calls'] > 3).astype(int)

combined = pd.concat([historical, new_data], ignore_index=True)
assert len(combined) <= 5000, "Exceeds retraining record limit!"
print(f"Combined retraining dataset: {combined.shape}")

FEATURES = [c for c in combined.columns if c != 'churn']
X, y = combined[FEATURES], combined['churn']
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42, stratify=y)

candidate_model = LogisticRegression(max_iter=1000, class_weight='balanced')
candidate_model.fit(X_train, y_train)
preds = candidate_model.predict(X_val)
candidate_f1 = f1_score(y_val, preds)
print(f"Candidate model — f1: {candidate_f1:.3f}")

prod_bundle = joblib.load('churn_model.joblib')
prod_model = prod_bundle['model']
prod_preds = prod_model.predict(X_val[prod_bundle['features']])
prod_f1 = f1_score(y_val, prod_preds)
print(f"Production model (v1) — f1 on same data: {prod_f1:.3f}")

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("Churn_Retraining")
with mlflow.start_run(run_name="candidate_v2"):
    mlflow.log_metric('candidate_f1', round(candidate_f1, 4))
    mlflow.log_metric('production_f1', round(prod_f1, 4))
    mlflow.sklearn.log_model(candidate_model, 'model')

if candidate_f1 > prod_f1:
    print("DECISION: Candidate model outperforms production. Promote to v2.")
    joblib.dump({'model': candidate_model, 'features': FEATURES, 'version': 'v2'},
    ↪ 'churn_model_v2.joblib')
else:
    print("DECISION: Keep current production model.")
```

Run: `python retrain_model.py`. Check the printed decision and view the new run in the MLflow UI (<http://localhost:5000>).

Step 3.2 — Upload the New Model and Deploy v2 (via GitHub + Cloud Run)

1. Upload `churn_model_v2.joblib` to **Cloud Storage** → **your-bucket-name** → **models/** via the Console (**Upload files**)
2. In your local project folder, replace `churn_model.joblib` with the contents of `churn_model_v2.joblib` (rename it to `churn_model.joblib` so `main.py` loads it without code changes)
3. Go to your GitHub repo → click **Add file** → **Upload files** → drag in the new `churn_model.joblib` → **Commit changes** (this overwrites the old one in the repo)
4. Cloud Run detects the GitHub commit automatically and starts a new build — go to **Cloud Run** → **churn-api** → **Revisions tab** and watch the new revision appear
5. Once deployed, open `your-service-url/docs` → **GET /metadata** → **Try it out** → **Execute** → confirm it now shows `model_version: v2`

Step 3.3 — Review Revisions and Roll Back (via Console)

1. Go to **Cloud Run** → **churn-api** → **Revisions tab** — you'll see a list of all revisions (v1 build, v2 build) with timestamps
2. Click **Manage Traffic** (top of the Revisions tab)
3. Set traffic to **100%** on the older (v1) revision, **0%** on the new (v2) revision
4. Click **Save**
5. Confirm the rollback: open `/docs` → **GET /metadata** → **Execute** → should show v1 again

► **Where to perform this — Cloud Run → churn-api → Revisions**

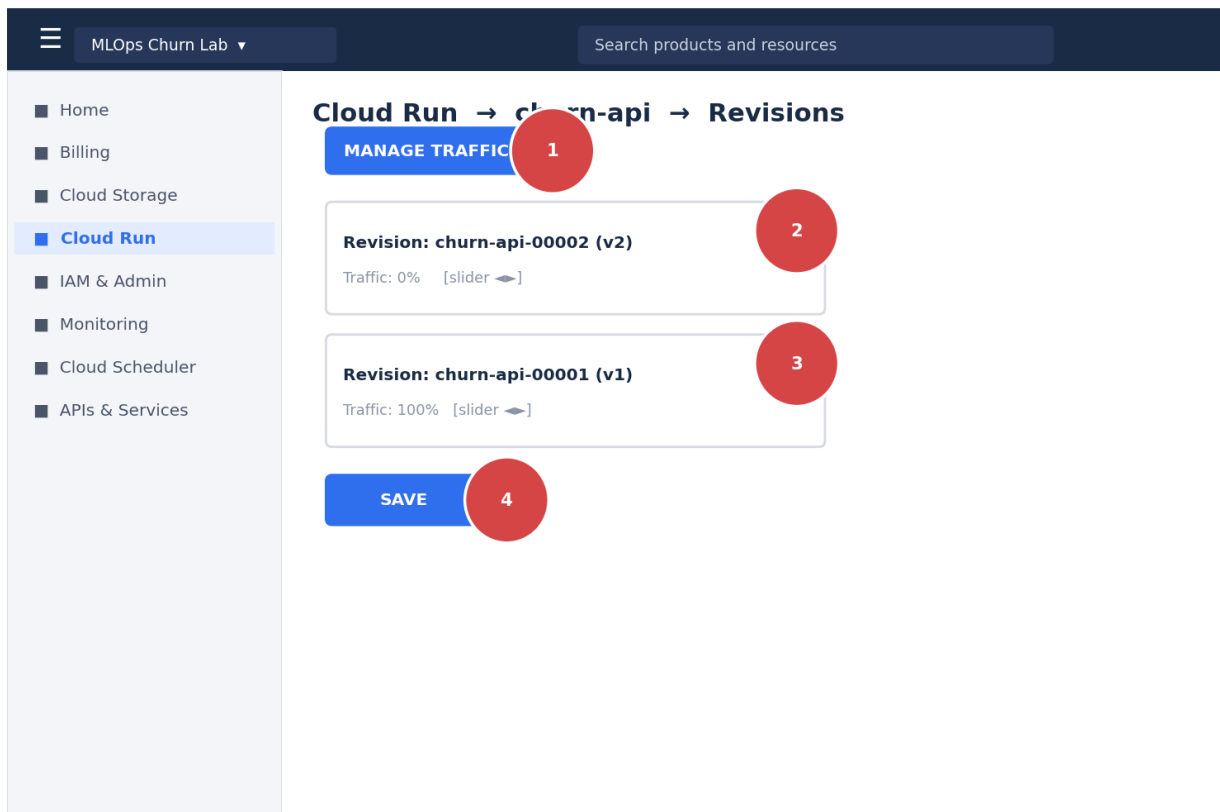


Figure 7: Revisions and Rollback wireframe

Discuss with learners: the same **Manage Traffic** screen supports splitting traffic between two revisions (e.g., 90% v1 / 10% v2) — this is a canary deployment, done entirely with sliders in the Console. Rolling deployments across multiple independent services typically need extra orchestration (e.g., a CI/CD pipeline) — mention this conceptually only.

Step 3.4 — Governance and IAM Review (via Console)

1. Go to **IAM & Admin** → **IAM** — view the full list of members and their roles on this project
2. Go to **IAM & Admin** → **Service Accounts** — view the default Cloud Run service account
3. Click **Create Service Account** → name it churn-viewer → **Create and Continue**
4. Assign role **Cloud Run Viewer** (read-only, cannot deploy) → **Continue** → **Done**
5. This demonstrates **least privilege**: this account can view services but never deploy or delete them
6. Go to **IAM & Admin** → **Audit Logs** — search for Cloud Run Admin API, show learners the automatic log of every deploy action taken today
7. Go back to **Cloud Run** → **churn-api** → **Revisions tab** — point out this itself is a built-in deployment/approval history, viewable by anyone with viewer access

► **Where to perform this — IAM & Admin** → **Service Accounts**

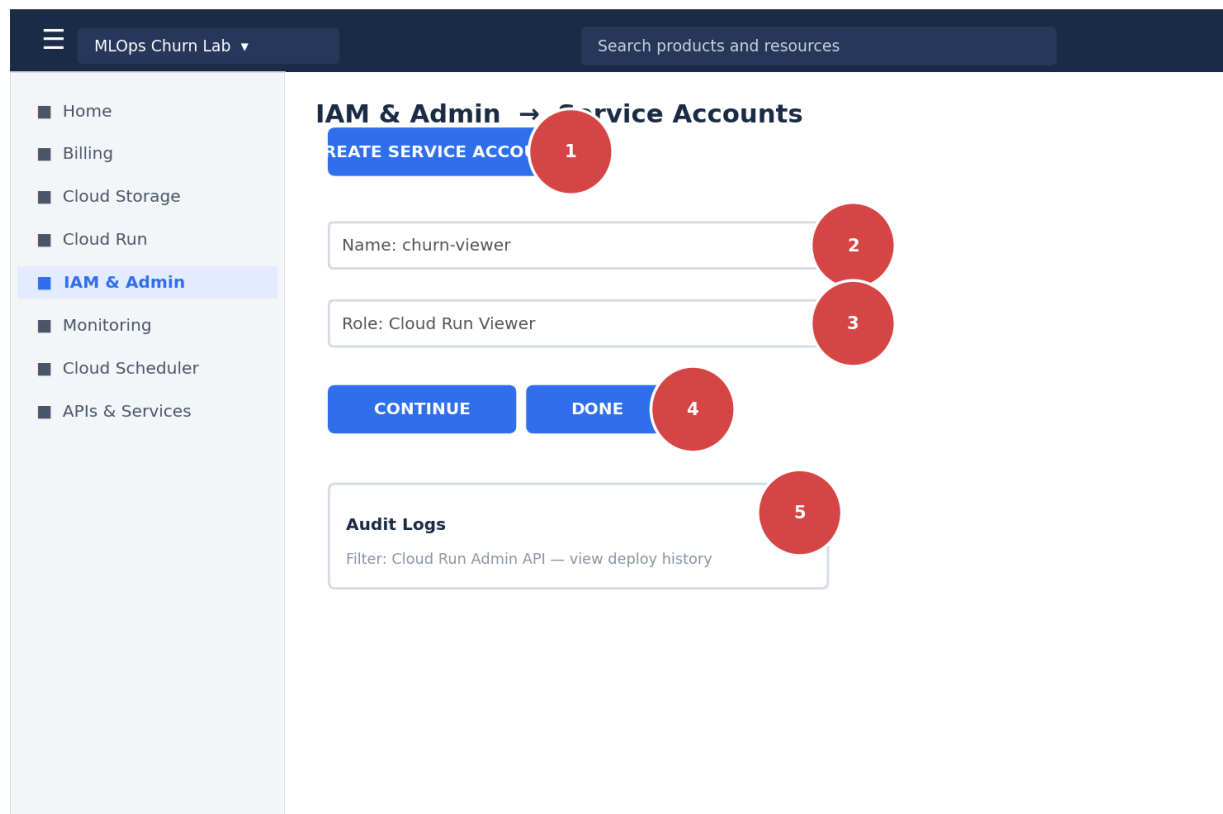


Figure 8: IAM Service Accounts wireframe

[DONE] Checkpoint — Section 3 complete when: v2 has been deployed and compared against v1 in MLflow, a rollback to v1 has been performed via **Manage Traffic**, and IAM/audit logs have been reviewed in the Console.

SECTION 4 — Cost, ROI, Automation, and Enterprise Architecture

Step 4.1 — Review Costs in Cloud Billing (Console)

1. Go to **Billing** → **Reports**
2. Filter by **Project: MLOps Churn Lab**
3. Group by **Service** — view the cost breakdown across Cloud Run, Cloud Storage, Artifact Registry, Cloud Build, Monitoring/Logging
4. Go to **Billing** → **Budgets & alerts** — confirm your budget from Step 1.2 is still active and check current spend against it

► **Where to perform this — Billing → Reports**

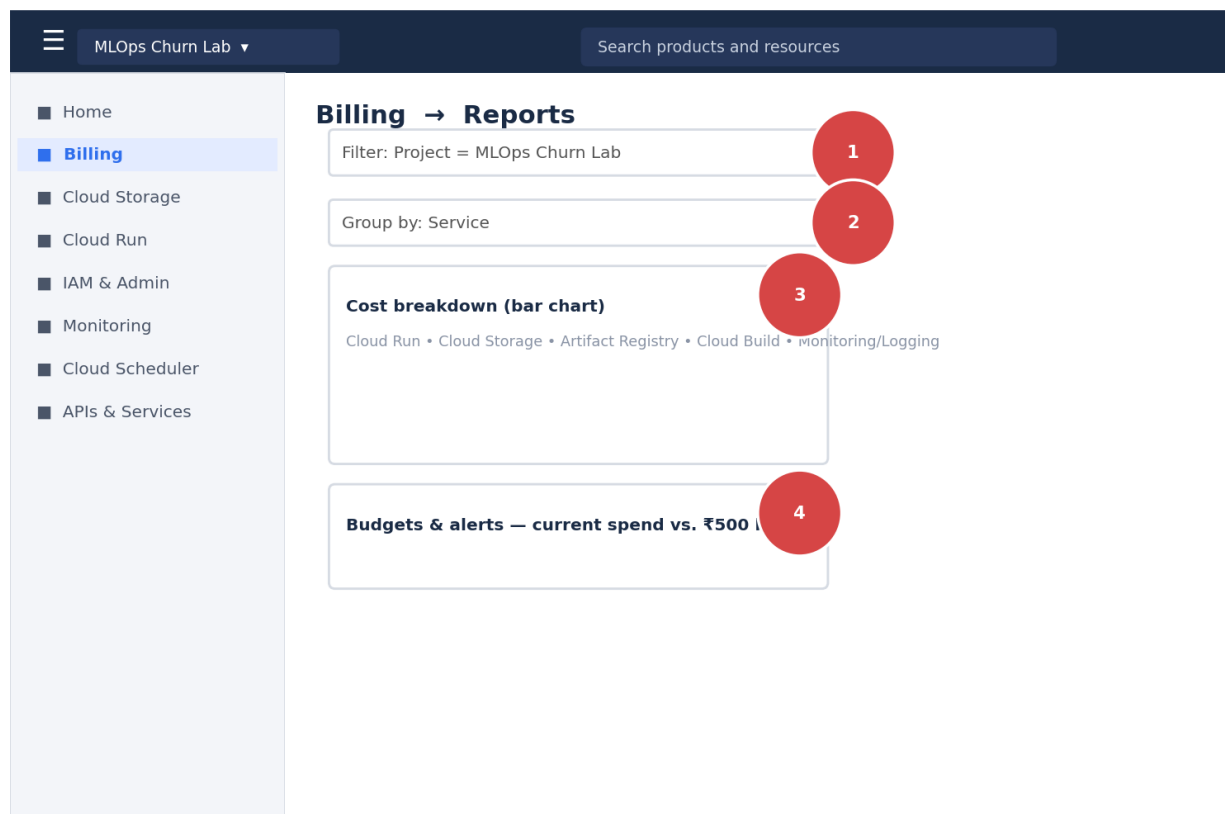


Figure 9: Billing Reports wireframe

Pair this screen with the `cost_and_roi.py` script output below to connect infra spend to business ROI (revenue saved, break-even customers).

Step 4.2 — Cost Estimate and ROI Calculation

`cost_and_roi.py` (run locally, walk through the output together):

```
# --- Illustrative cost model (asia-south1, approximate, for teaching) ---
CLOUD_RUN_VCPU_SEC = 0.000024
CLOUD_RUN_MEM_GB_SEC = 0.0000025
```

```
STORAGE_GB_MONTH = 0.02
REQUESTS_PER_DAY = 1000
AVG_REQUEST_SECONDS = 0.2

daily_compute_cost = REQUESTS_PER_DAY * AVG_REQUEST_SECONDS * (CLOUD_RUN_VCPU_SEC
↪ + 2 * CLOUD_RUN_MEM_GB_SEC)
monthly_compute_cost = daily_compute_cost * 30
monthly_storage_cost = 0.5 * STORAGE_GB_MONTH
monthly_total = monthly_compute_cost + monthly_storage_cost
print(f"Estimated monthly compute cost: ${monthly_compute_cost:.2f}")
print(f"Estimated monthly storage cost: ${monthly_storage_cost:.2f}")
print(f"Estimated total monthly cost:  ${monthly_total:.2f}")

# --- Business ROI ---
NUM_CUSTOMERS_SCORED = 1000
CHURN_RATE_BASELINE = 0.25
RETENTION_SUCCESS_RATE = 0.30
AVG_CUSTOMER_LIFETIME_VALUE = 150
COST_PER_RETENTION_OFFER = 10

at_risk = NUM_CUSTOMERS_SCORED * CHURN_RATE_BASELINE
retained = at_risk * RETENTION_SUCCESS_RATE
revenue_saved = retained * AVG_CUSTOMER_LIFETIME_VALUE
intervention_cost = at_risk * COST_PER_RETENTION_OFFER
net_benefit = revenue_saved - intervention_cost - monthly_total
roi_pct = (net_benefit / (intervention_cost + monthly_total)) * 100
breakeven = (intervention_cost + monthly_total) / (AVG_CUSTOMER_LIFETIME_VALUE *
↪ RETENTION_SUCCESS_RATE)

print(f"\nAt-risk customers identified: {at_risk:.0f}")
print(f"Customers retained (est.):      {retained:.0f}")
print(f"Revenue saved:                  ${revenue_saved:.2f}")
print(f"Total cost (ops+outreach):       ${intervention_cost + monthly_total:.2f}")
print(f"Net monthly benefit:             ${net_benefit:.2f}")
print(f"ROI:                             {roi_pct:.1f}%")
print(f"Break-even at-risk customers: {breakeven:.0f}")
```

Run: `python cost_and_roi.py`. Discuss each line as a group — this is where the technical build connects to a number a business stakeholder cares about.

Step 4.3 — Automate Scheduled Predictions (Console)

You already added a `/run-batch-predict` endpoint in Step 1.12. Now schedule it:

1. Go to **Cloud Scheduler** → **Create Job**
2. Name: `churn-daily-predictions`
3. Region: **asia-south1**
4. Frequency (cron): `0 6 * * *` (6 AM daily)
5. Timezone: `Asia/Kolkata`
6. Target type: **HTTP**
7. URL: `your-service-url/run-batch-predict`
8. HTTP method: **POST**
9. Auth header: **Add OIDC token**, service account: the default Compute service account (needed since the endpoint requires authentication if you locked it down; if the service allows unauthenticated calls, you can leave auth off)

10. Click **Create**

Test it immediately without waiting until 6 AM:

1. Go to **Cloud Scheduler**, find churn-daily-predictions in the job list
2. Click the [...] menu → **Force run**
3. Go to **Cloud Run** → **churn-api** → **Logs** — confirm the request came through
4. Go to **Cloud Storage** → **your-bucket-name** → **predictions/** — confirm a new times-tamped CSV appeared

► **Where to perform this — Cloud Scheduler → Create Job**

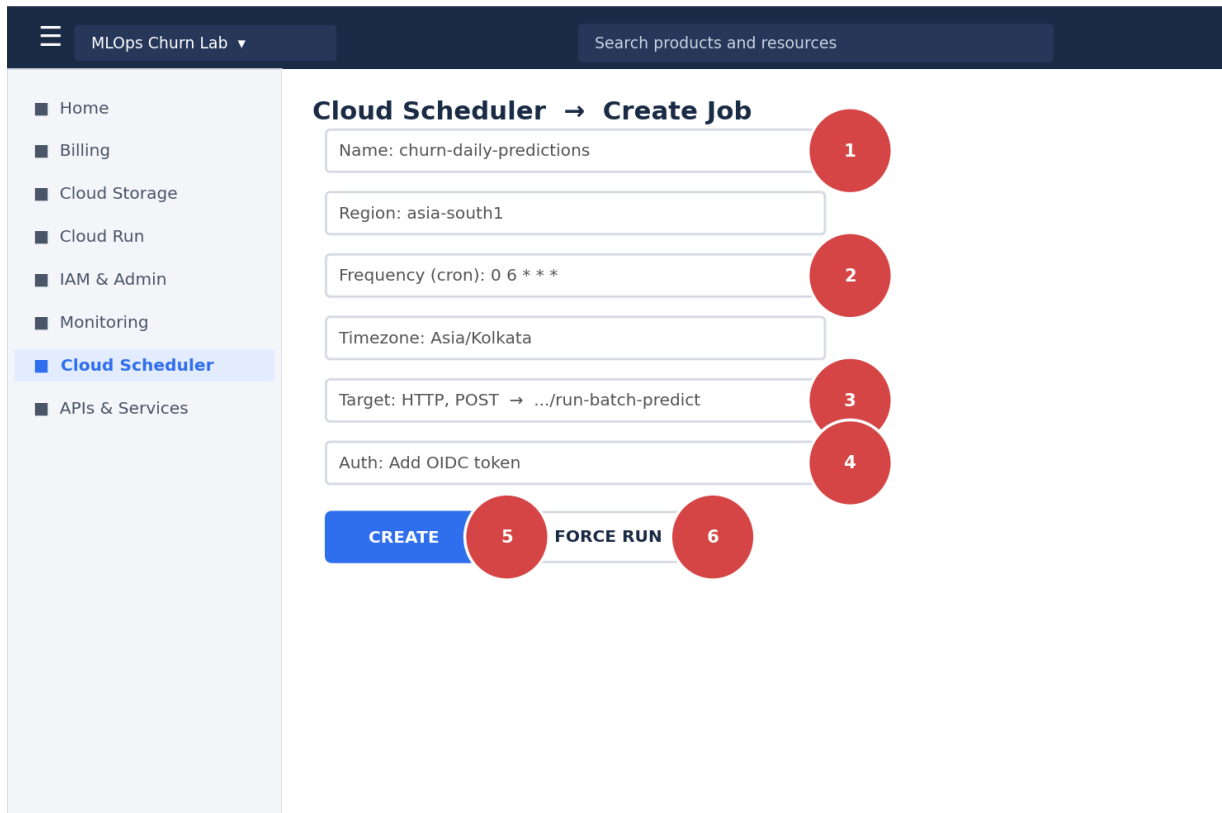


Figure 10: Cloud Scheduler wireframe

Discuss conceptually (no hands-on): Cloud Workflows and Pub/Sub for multi-step event-driven pipelines (found under their own sections in the Console left sidebar), Eventarc for triggering on GCS file uploads, Cloud Build triggers for full CI/CD, Vertex AI Pipelines for managed ML orchestration — show learners where each lives in the Console menu without configuring them.

Step 4.4 — Present the Full Architecture

Show this diagram on screen as the closing summary:

Business Problem

- > Historical Data -> Model (Logistic Regression)
- > Cloud Storage (training-data/, models/)
- > FastAPI -> Docker -> GitHub -> Cloud Build -> Cloud Run

- > Predictions -> Cloud Storage (predictions/)
- > Monitoring (Cloud Logging + Monitoring) -> Drift Detection (Evidently)
- > Experiment Tracking (MLflow) -> Retraining -> Model Versioning (v1 -> v2)
- > Governance (IAM, audit logs) -> Automation (Cloud Scheduler)
- > Business Evaluation (Cost + ROI)

[DONE] Checkpoint — Section 4 complete when: the Billing report has been reviewed, cost/ROI numbers have been discussed, and the Cloud Scheduler job has successfully triggered a batch prediction visible in Cloud Storage.

SECTION 5 — Conclusion, Cleanup, and Q&A

Step 5.1 — Recap Checklist

- ☐ Project, billing, budget, and alerts configured (Console)
- ☐ Model trained locally, code pushed to GitHub, deployed via Cloud Run's "deploy from repository" wizard
- ☐ Predictions generated via `/run-batch-predict` and stored in Cloud Storage with timestamp + version
- ☐ Cloud Monitoring dashboard and alert created and reviewed
- ☐ Drift detected and compared between a healthy and a drifted batch (HTML reports in logs/)
- ☐ Experiments tracked and viewed in the MLflow web UI
- ☐ Model retrained, compared against production, promoted to v2 via GitHub commit
- ☐ Rollback performed using **Manage Traffic** in Cloud Run
- ☐ IAM roles and audit logs reviewed in Console
- ☐ Cost reviewed in Billing Reports; ROI and break-even calculated
- ☐ Cloud Scheduler job successfully triggered a prediction run

Step 5.2 — Mandatory Cleanup (via Console — Do Not Skip)

1. **Cloud Scheduler** → select churn-daily-predictions → **Delete**
2. **Cloud Run** → select churn-api → **Delete**
3. **Cloud Build** → **Triggers** → delete the auto-created GitHub trigger
4. **Artifact Registry** → delete the repository that Cloud Build created for your images
5. **Cloud Storage** → select your-bucket-name → **Delete** (confirm you no longer need the contents first)
6. **Monitoring** → delete the alert policy and dashboard created in Step 2.1
7. Optional: go to **IAM & Admin** → **Settings** → **Shut down project** if this project was created solely for the lab

Step 5.3 — Open Q&A

Close by reinforcing: **enterprise MLOps is a continuous lifecycle** — technical performance, operational reliability, governance, cost, automation, and business value all have to be managed together, and every single one of those pieces was just demonstrated using nothing but the Google Cloud Console.